

Week 03: Typescript

Agenda

- JS & TS
- TypeScript Config
- TypeScript Info
- TypeScript Type System
- Callbacks, Promises
- Data Validation with Zod
- Demo/Assignment

JavaScript Overview

- Scripting language run by Engine as JIT (Just-in-time) compiler
- Runs in Web Browsers and standalone (NodeJS, Deno, Bun)
- Allows frontend and backend development
- Single-threaded, Dynamic, Multi-paradigm, Supporting OOP, Imperative and Declarative styles
- Type coercion, Falsy values, Compatibility and Number implementation are problems of JavaScript

What is TypeScript?

- A **superset** of JavaScript
- Adds **static typing** and other enhancements
- Developed by **Microsoft**
- Needs to be **transpiled** into JavaScript
- Used for **frontend & backend** development

TypeScript Config (1/3) - Basics

```
{
  "compilerOptions": {
    "target": "ESNext", // Transpile to ESNext JavaScript
    "strict": true, // Enable strict type checking
    "module": "CommonJS" // Specify module system (Prefer ESM in modern projects)
  }
}
```

- Defines **compiler options**
- Specifies **JavaScript version** for output
- Enforces **strict mode** for better type safety

TypeScript Config (2/3) - Key Options

```
{
  "compilerOptions": {
    "lib": ["ESNext", "DOM"], // Include ESNext and DOM libraries
    "outDir": "./dist", // Output directory for compiled files
    "rootDir": "./src", // Root directory for TypeScript files
    "sourceMap": true // Enable source maps for debugging
  }
}
```

- `lib` : Defines built-in libraries available
- `outDir` : Specifies output directory for compiled files
- `rootDir` : Specifies root directory for TypeScript files
- `sourceMap` : Enables source maps for debugging

TypeScript Config (3/3) - Strict Options

```
{
  "compilerOptions": {
    "noImplicitAny": true, // Disallow implicit any types
    "strictNullChecks": true, // Enforce strict handling of null/undefined
    "alwaysStrict": true // Always enable strict mode
  }
}
```

- `noImplicitAny` : Prevents variables from implicitly becoming `any`
- `strictNullChecks` : Ensures strict handling of `null` and `undefined`
- `alwaysStrict` : Ensures strict mode is always enabled

Basic Types & Syntax

```
let isDone: boolean = false; // Boolean type
let age: number = 22; // Number type
let name: string = "Alice"; // String type
let list: number[] = [1, 2, 3]; // Array of numbers
let tuple: [string, number] = ["hello", 10]; // Tuple type
```

- `any` (disables type checking, **not recommended**)
- `unknown` (safer alternative to `any`, requires type checking)
- `void`, `undefined`

Any vs Unknown Type

```
let valueAny: any;  
valueAny = 42;  
valueAny.toFixed(); // ✅ No error (but unsafe)  
  
let valueUnknown: unknown;  
valueUnknown = "Hello";  
// valueUnknown.toUpperCase(); ❌ Error: Object is of type 'unknown'  
if (typeof valueUnknown === "string") {  
  console.log(valueUnknown.toUpperCase()); // ✅ Now it's safe  
}
```

- `any`: Allows **any** type, bypassing type safety (**use with caution**)
- `unknown`: More **strict**, requires type **narrowing** before use

Functions & Typing

```
function greet(name: string): string {  
  return `Hello, ${name}!`; // Function with return type string  
}  
  
const add = (a: number, b: number): number => a + b; // Arrow function with typed parameters
```

- Optional parameters (`?`)
- Spreading parameters (`...args`)

Union & Intersection Types

Union Types

```
type StringOrNumber = string | number;  
let value: StringOrNumber;  
value = "Hello";  
value = 42;  
// value = true; ❌ Error
```

- A value can be one of multiple types

Intersection Types

```
type Person = { name: string };  
type Employee = { id: number };  
type EmployeePerson = Person & Employee;  
  
const emp: EmployeePerson = { name: "John", id: 123 };
```

- Combines multiple types into one

Destructuring & Rest/Spread Syntax

Destructuring

```
const user = { name: "Alice", age: 25 };  
const { name, age } = user;  
console.log(name, age); // "Alice", 25
```

- Extracts values from objects/arrays

Rest Syntax

```
const [first, ...rest] = [1, 2, 3, 4];  
console.log(first); // 1  
console.log(rest); // [2, 3, 4]
```

- Gathers remaining elements into an array

Spread Syntax

```
const obj1 = { a: 1, b: 2 };  
const obj2 = { ...obj1, c: 3 };  
console.log(obj2); // { a: 1, b: 2, c: 3 }
```

- Expands elements from an object

Type Guards & Type Assertions

Type Guards

```
function checkType(value: unknown) {  
  if (typeof value === "string") {  
    console.log("It's a string!");  
  } else if (value instanceof Date) {  
    console.log("It's a Date object!");  
  } else if ("length" in (value as any)) {  
    console.log("It has a length property!");  
  }  
}
```

- `typeof` : Checks primitive types
- `instanceof` : Checks object instances
- `in` : Checks if a property exists

Callbacks & Promises

Callbacks

```
function fetchData(callback: (data: string) => void) {  
  setTimeout(() => {  
    callback("Data loaded");  
  }, 2000);  
}  
  
fetchData((data) => console.log(data)); // Output after 2 seconds: "Data loaded"
```

- Functions **passed as arguments** to handle asynchronous operations
- Can lead to **callback hell** (nested functions difficult to read)

Promises

```
function fetchData(): Promise<string> {  
  return new Promise((resolve) => {  
    setTimeout(() => resolve("Data loaded"), 2000);  
  });  
}  
  
fetchData().then((data) => console.log(data)); // Output after 2 seconds: "Data loaded"
```

- `Promise` represents an **async operation**
- `.then()` handles **success**, `.catch()` handles **errors**
- Replaces callback-based syntax for **better readability**

Async/Await

```
async function fetchDataAsync() {  
  const data = await fetchData();  
  console.log(data); // Output after 2 seconds: "Data loaded"  
}  
  
fetchDataAsync();
```

- **Syntactic sugar** for Promises
- Reads like **synchronous code**, but executes asynchronously

Data Validation with Zod

- **Zod** is a TypeScript-first schema declaration and validation library.
- It helps validate runtime data (e.g., from an API or user input) against a defined schema.
- Unlike TypeScript types which only exist at compile time, Zod works at runtime.

Basic Zod Usage

```
import { z } from "zod";

// Define a schema
const UserSchema = z.object({
  name: z.string(),
  age: z.number().min(18),
});

// Extract the TypeScript type
type User = z.infer<typeof UserSchema>;

// Validate data
const result = UserSchema.parse({ name: "Alice", age: 25 }); // 
// const result2 = UserSchema.parse({ name: "Bob", age: 15 }); //  Throws error
```

"The only way to learn a new programming language is by writing programs in it."

— Dennis Ritchie

 Now, let's code! 